Introduction to SQL
— Chapter 3

2025 年 9 月

北京郵電大學

3.1 Overview of SQL

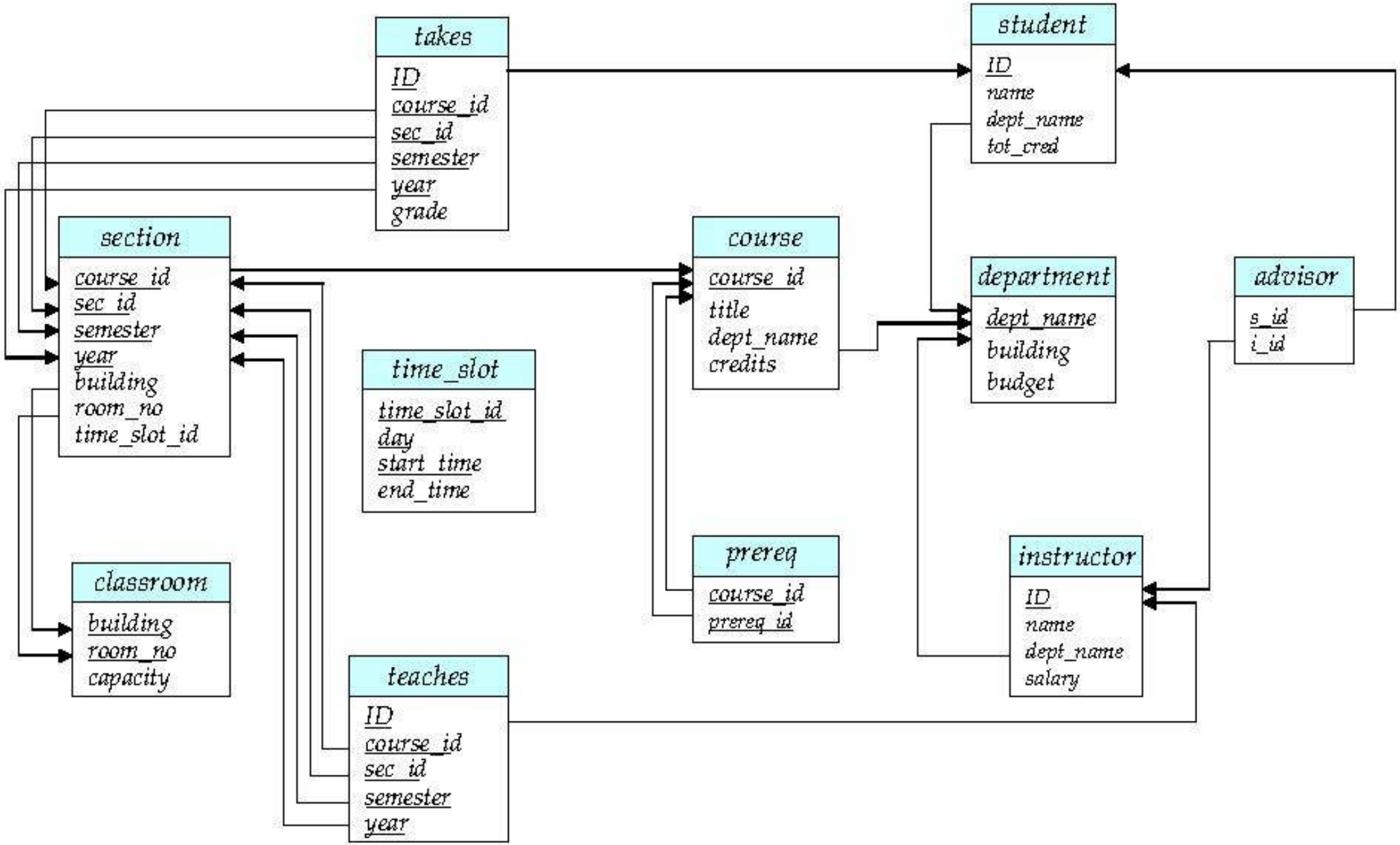
- **S**tructured **Q**uery **L**anguage
  - **Widely** used query language
  - **Extended** into other areas, e.g. BigData
  - **Not all** examples work on our **particular** system.

SQL Parts

- DML -- **Data Manipulation Language**
  - The ability to **query/retrieve** information
  - The ability to **select, insert , delete, update** tuples
- DDL -- **Data Definition Language**
  - **define, drop, modify** on schemas, views and index
- DDL: **integrity**
  - commands for specifying integrity constraints
- DDL: **View definition**
  - commands for defining views
- **Transaction control**
  - commands for specifying the beginning and ending of transactions.
- **Embedded SQL and dynamic SQL**
  - statements embedded within programming languages.
- **Authorization**

data query	Select
data manipulation	Insert, Delete, Update
data definition	Create, Drop, Alter (on schema)
data control	Grant, Revoke
transaction processing	begin transaction, commit, rollback
指针 / 游标控制语言 (CCL)	DECLARE CURSOR ， FETCH INTO 和 UPDATE WHERE CURRENT

University Database



3.2 SQL Data Definition

Def. DDL allows **specification** about relations ：

- The **schema** for each relation.
- The **type of values** associated with each attribute.
- The **integrity constraints**, e.g. age>0
- The **indices** to be maintained for each relation.
- The **security and authorization** information for each relation
- The **physical storage structure** of each relation on disk

Domain Types

- **char(n).** Fixed length character string, user-specified length *n*.
- **varchar(n).** Variable length character string, user-specified length *n*.
- **int.** Integer (the integers that is machine-dependent).
- **smallint.** Small integer (machine-dependent subset of the integer).
- **numeric(p,d).** Fixed point number, user-specified precision of *p* digits, with *d* digits to the right of decimal point.
  (ex., **numeric**(3,1), allows 44.5 to be stores exactly)

Domain Types in SQL

- **date.** Dates, containing a (4 digit) year, month and date
  - e.g. **date** '2001-07-27'
- **time.** Time of day, in hours, minutes and seconds.
  - e.g. **time** '09:00:30' **time** '09:00:30.75'
- **timestamp**: date plus time of day
  - e.g. **timestamp** '2001-7-27 09:00:30.75'
- **Interval**: period of time
  - Subtracting a date/time/timestamp value from another gives an interval value
    - e.g. Interval '1' day
    - Interval values can be added to date/time/timestamp values
- **Null** values allowed in all domain types

  Declaring an attribute to be **not null** prohibits null values for that attribute.
- **create domain** construct creates **user-defined domain** types 自定义类型
  - **create domain** *person-name* **char**(20) **not null**

注意

- 在 Oracle 等大型商用数据库系统中，关系表的属性的名字只能取英文名，不支持中文属性名
- SQL Server 支持中英文属性名
- 开发大型数据库应用时，程序所访问的关系表的属性名字（最好）取为英文名，便于应用程序的可移植性！
  - 准确表示英文专业术语（英文属性名称), e.g. 分组 packet

Basic Schema Definition: Create Table

- **Def.** An SQL relation is defined using the **create table** command:

  create table *r*

  (A_1 D_1, A_2 D_2, ..., A_n D_n,
  (integrity-constraint$_1$), ...,
  (integrity-constraint$_k$))

  - *r* is the name of the relation
  - A_i is an attribute name in the schema of relation *r*
  - D_i is the data type of values in the domain of attribute A_i

- Example:

  create table *instructor* (
  ID **char**(5),
  name **varchar**(20),
  dept_name **varchar**(20),
  salary **numeric**(8,2))

Integrity Constraints in Create Table

- Types of integrity **constraints**
  - **primary key** (A_1, ..., A_n)
  - **foreign key** (A_m, ..., A_n) **references** *r*
  - **not null**
- **Prevents** update that violates integrity constraint.
- Example:

```
create table instructor (
        ID              char(5),
        name            varchar(20) not null,
        dept_name       varchar(20),
        salary          numeric(8,2),
        primary key (ID),
        foreign key (dept_name) references department);
```

More Relation Definitions

- **create table** *student* (
  ID **varchar**(5),
  name **varchar**(20) not null,
  dept_name **varchar**(20),
  tot_cred **numeric**(3,0),
  primary key *(ID),*
  foreign key *(dept_name*) **references** *department)*;

- **create table** *takes* (
  ID **varchar**(5),
  course_id **varchar**(8),
  year **numeric**(4,0),
  grade **varchar**(2),
  primary key *(ID, course_id, year)* ,
  foreign key (*ID)* **references** *student,*
  foreign key (*course_id, year)* **references** *section*);

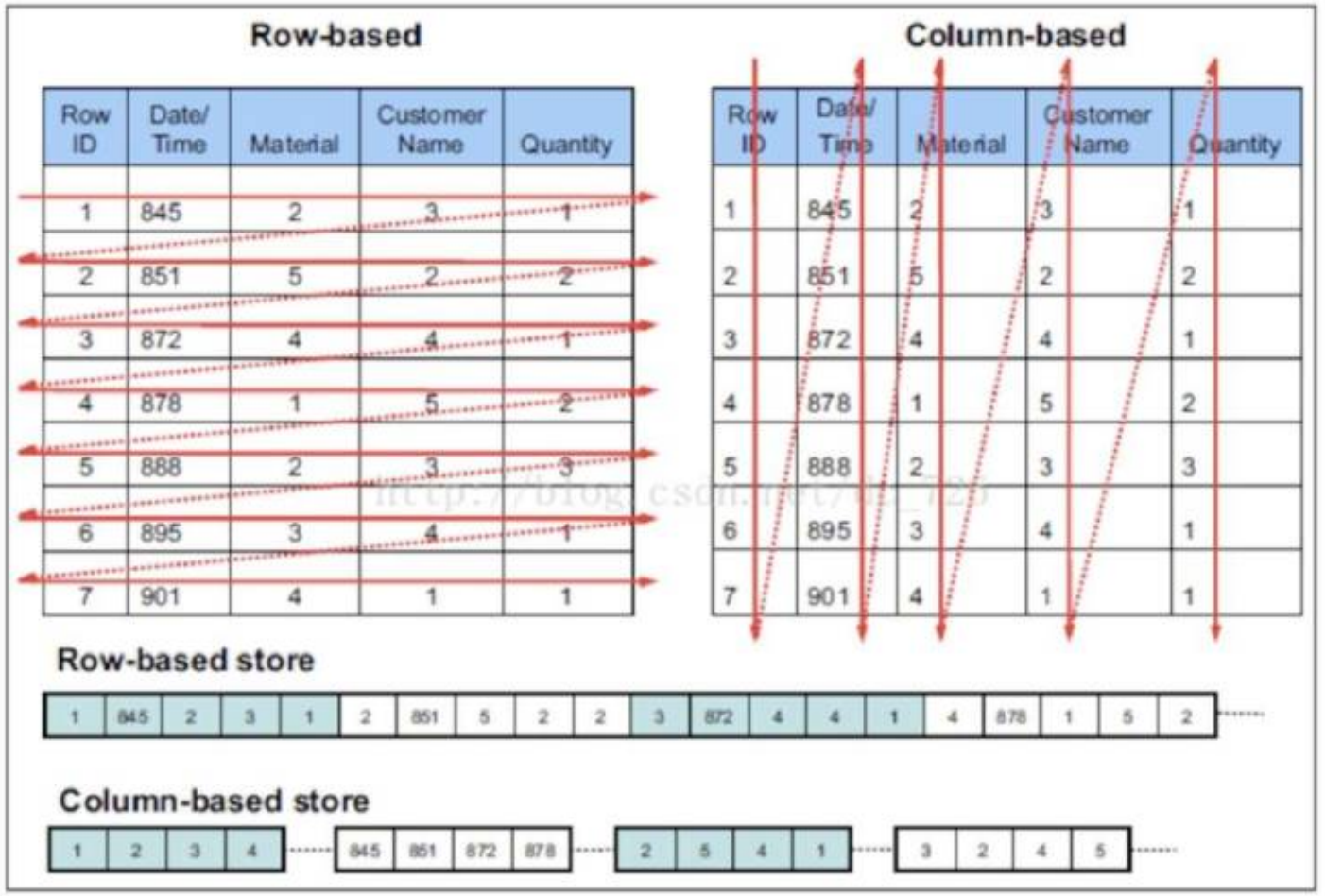
Updates to Schemas

- **Drop Table**
  - **drop table** *r*
- **Alter**
  - **alter table** *r* **add** *A D*
    - where *A* is the name of attribute added to *r* and *D* is the domain of *A*.
    - All exiting tuples are assigned *null* as the value for the added attribute
    - e.g. **alter table *student* add *age* integer**
  - **alter table** *r* **drop** *A*
    - where *A* is the name of an attribute of *r*
    - e.g. **alter table *student* *drop* *age***
    - Dropping of attributes not supported by many databases

Updates to Schemas

- The tuples/rows in **table** are stored **row by row** (按行存储).
- **adding** attributes or **dropping** the table leads to **costly modifying** the records and then **moving** these records

- **NoSQL DB**
  - **BigData**
  - **store data column by column**



3.3 Basic Structure of SQL Queries

- A typical SQL query has the form:

select $A_1, A_2, ..., A_n$
from $r_1, r_2, ..., r_m$
where P

query result :

	A_1	A_2	…	…	A_n
t_1	*	*	*	*	*
t_2	*	*	*	*	*
	*	*	*	*	*
	*	*	*	*	*
t_k	*	*	*	*	*

  - A_i represents an attribute
  - r_i represents a relation
  - P is a predicate.

- The result of an SQL query is a relation
- This query is equivalent to the relational algebra expression

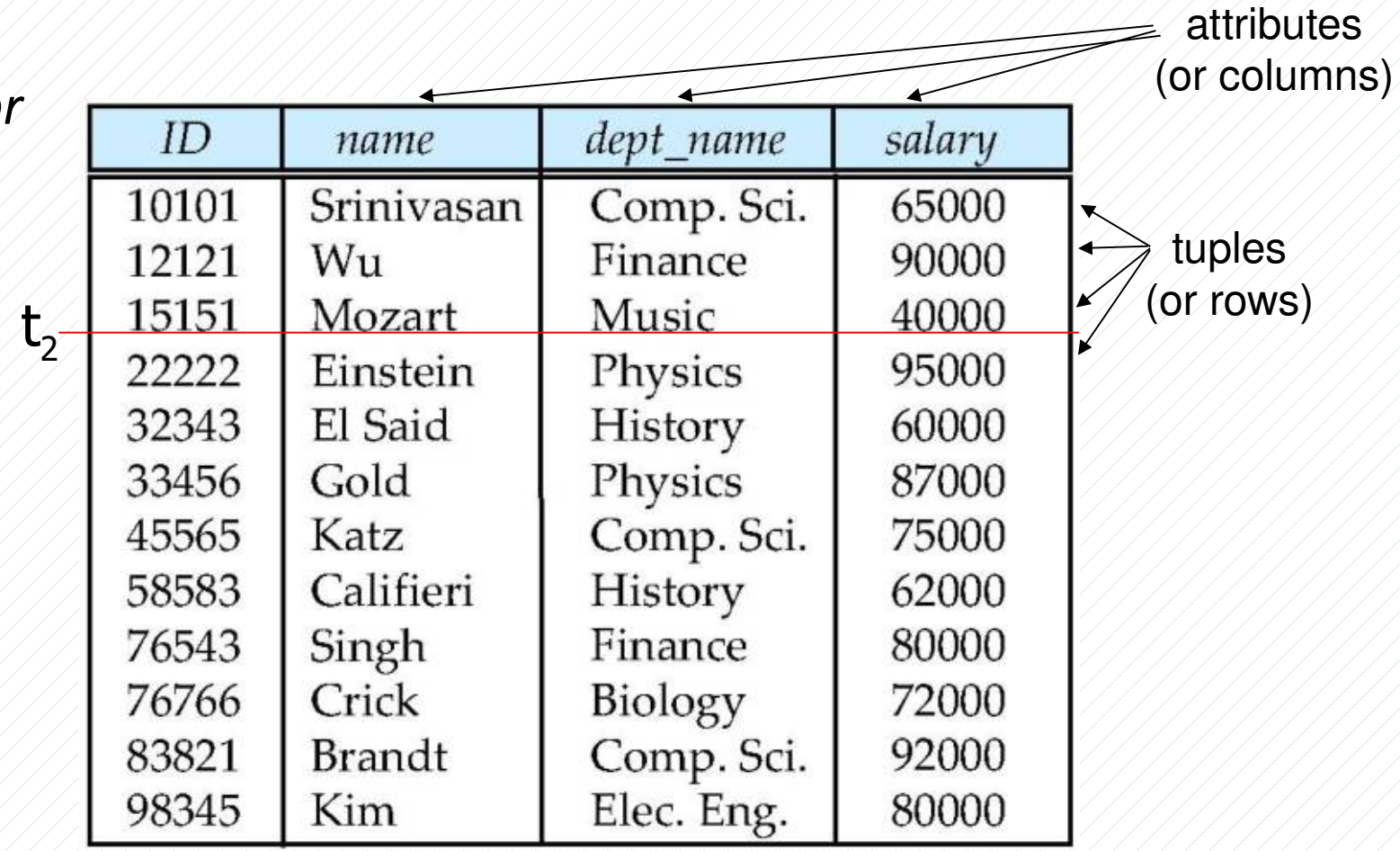
$$\Pi\Pi_{A1, A2, ..., An}(\sigma_P (r_1 \times r_2 \times ... \times r_m))$$

1. The select Clause

- The **select** clause lists the **attributes**
  - corresponds to **projection operation** of relational algebra
- Example: find the names of all instructors:

select *name*

from *instructor*



ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

- NOTE: names are **case insensitive** (upper- or lower-case letters.)
  - *Name* ≡ *NAME* ≡ *name*

The select Clause

- SQL allows **duplicates** in relations.
- To force elimination of duplicates, **insert** the keyword **distinct.**
  - Find the department names of all instructors, and remove duplicates

```
select distinct dept_name
from instructor
```

- The keyword **all** specifies that duplicates should not be removed.

```
select all dept_name
from instructor
```

The select Clause (Cont.)

- An **asterisk** in the select clause denotes all attributes

  select *
  from *instructor*

实际应用中避免在 select 子句中使用 * ，防止索引失效

- An attribute can be a literal with no **from** clause

  select '437'

  - Results is a table with one column and a single row with value 437

437

  - Can give the column a name using:

  select '437' **as** *FOO*

FOO
437

- An attribute can be a literal with **from** clause

  select 'A'
  from *instructor*

A
A
…
A
A

  - Result is a table with one column and *N* rows , each row with value “A”
  - (number of tuples in *instructors*)

The select Clause (Cont.)

- The **select** clause can contain arithmetic expressions involving **operation**, +, –, *, and /, and **operating** on constants or attributes of tuples.
  - The query:

    select *ID, name, salary/*12
    from *instructor*

    would return a relation that is the same as *instructor*, except that the value of *salary* is divided by 12.
  - **Rename** "salary/12" using the **as** clause:

    select ID, name, salary/12 **as** monthly_salary

2. The where Clause

- The **where** clause specifies conditions that the result must satisfy
  - corresponds to **selection predicate** of relational algebra.
  - allows the **logical connectives, and, or,** and **not**

➢ Find all instructors in Comp. Sci. dept

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- **Logical connectives** involve **comparison operators** <, <=, >, >=, =, and <>.

➢ Find all instructors in Comp. Sci. with salary > 7000

```
select name
from instructor
where dept_name = 'Comp. Sci.'  and salary > 70000
```

Where Clause Predicates

- **Between comparison operator**
  - Find names of all instructors with salary between $90,000 and $100,000 (≥ $90,000 and ≤ $100,000)
    - **select** *name*
      from *instructor*
      where *salary* **between** 90000 **and** 100000
- **Tuple comparison operator**
    - **select** *name*, *course_id*
      from *instructor*, *teaches*
      where (*instructor.ID*, *dept_name*) = (*teaches.ID*, 'Biology');

The from Clause

- The **from** clause lists relations
  - corresponds to **Cartesian product** operation of relational algebra.
- Find Cartesian product *instructor X teaches*

  select ∗
  from *instructor, teaches*

  - generates every **instructor–teaches pair**, with all attributes from both relations.
  - For **common** attributes (*ID*), attributes in resulting table are renamed using the relation name (*instructor.ID*)

➢ Cartesian product not very useful directly, but useful with where-clause condition

3. Cartesian Product

instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Inst.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Pinance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Pinance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Pinance	90000	22222	PHY-101	1	Fall	2009
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…

Examples

- Find names of all instructors who have taught some course and the course_id
  - **select** *name, course_id*
    from *instructor , teaches*
    where *instructor.ID = teaches.ID*

- Find names of all instructors in Art department who have taught some course and the course_id
  - **select** *name, course_id*
    from *instructor , teaches*
    where *instructor.ID = teaches.ID*
    and *instructor. dept_name =* 'Art'

注意事项

- 子句 **from** 包括多个关系表
  - 要在 where 子句中加入连接条件
  - 防止出现 costly 多表笛卡尔积操作
- 频繁执行 SQL 查询，**from** 子句表的个数不要过多！
  - 避免耗时费力的多表连接操作
  - 不要超过 4 个表
  - 如果频繁执行查询涉及 $N \geq 4$ 张表，将这 N 张表的数据进行合并

4. natural join

- **natural join** in ***from*** subclause
  - Find names of all instructors in Art who have taught some course and course_id

select *name, course_id*
from *instructor* ***natural join*** *teaches*
where *instructor. dept_name* = 'Art'

- Compared with:

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*
and *instructor. dept_name* = 'Art'

SQL Syntax

Select {ALL | DISTINCT } {<column expression1> }

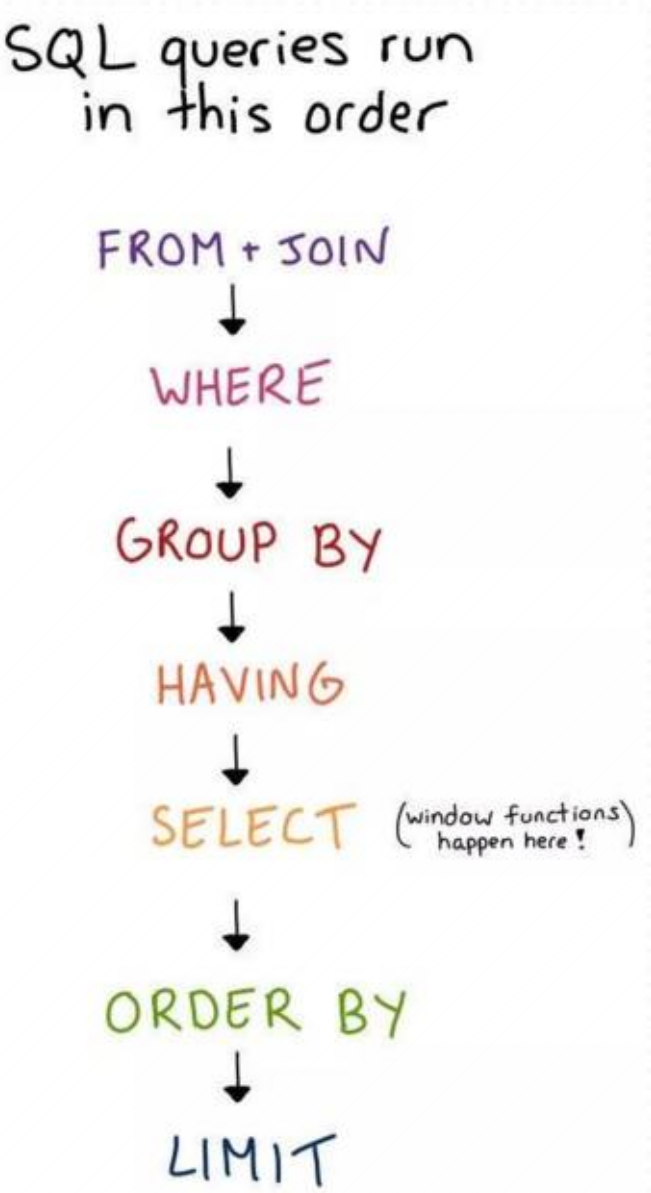
From {<table_name or view_name> }

{ **Where** < conditional expression 1 > }

{ **Group by** < column_name 1 >

Having{ < conditional expression 2 > } }

{ **Order by** <column_name 2 > { ASC | DESC } }



Computation Procedure of Select

- **1. Selecting** tuples that satisfy conditions **defined** by *where* sub_clause **from** base tables or views **listed** in *from* sub_clause
  - selecting attributes from these tuples, and forming a **result table**
- **2. Grouping** the resulting table on conditional expression 2,
  - on each group, **conducting** group/aggregate function
  - outputting each group as results.
- **3. Having** the resulting group,
  - outputting group that satisfies column_name 1 as results.
- **4. Sorting** the resulting table or groups in descending or ascending orders.

3.4 Additional Basic Operation ： The Rename Operation

- **renaming relations** and **attributes** using the **as** clause:

  old-name **as** *new-name*

- **Find** names of all instructors who have a higher salary than *instructor* in Comp. Sci.
  - **select distinct** *T.name* ***as TeacherName***

    from *instructor* **as** *T*, *instructor* **as** *S*　/***tuple variable** ，元组变量

    where *T.salary* > *S.salary* **and** *S.dept_name* = *'Comp. Sci.'*

Keyword **as** is optional and omitted:
instructor **as** *T* ≡ *instructor T*

T

T.name　　*T.salary*

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

S

S.name　*S.dept_name*　*S.salary*

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

return

Tuple Variables

- 涉及到对 *instructor* 关系中属性 *salary* 的不同值的比较
  - **select distinct** *T.name*

    from *instructor* **as** *T, instructor* **as** *S*

    where *T.salary > S.salary* **and** *S.dept_name = 'Comp. Sci.'*

    - /* 利用 T 和 S 区分不同的 instructor!，实现了对同一属性的不同值的比较

String Operations

- **String-matching operator** for comparisons on **character strings**.
  The operator **like** uses patterns two special characters:
  - percent (%). The % character matches any substring.
  - underscore (_). The _ character matches any character.
- Find the names of all instructors whose name includes the substring “dar”.

```
select name
from instructor
where name like '%dar%'
```

- Match the string “100%”

```
like '100 \%'  escape  '\'
```

in that above we use backslash (\) as the escape character.

String Operations (Cont.)

- Patterns are case sensitive.
- Pattern matching examples:
  - 'Intro%' matches any string beginning with “Intro”.
  - '%Comp%' matches any string containing “Comp” as a substring.
  - '_ _ _' matches any string of exactly three characters.
  - '_ _ _ %' matches any string of at least three characters.
- A variety of string operations such as
  - concatenation (using “||”)
  - converting from upper to lower case (and vice versa)
  - finding string length
  - extracting substrings, etc.

Attribute Specification in the Select Clause

- * denotes all attributes

- **select** *instructor.**

  from *instructor, teaches*

  where *instructor.ID = teaches.ID*

Ordering the Display of Tuples

- **List** in alphabetic order the names of all instructors

```
select distinct name
        from    instructor
        order by name
```

- **Specify desc** for descending order or **asc** for ascending order
  - ascending order is the default.
    - Example: **order by** *name* **desc**
- **Sort** on multiple attributes
  - Example: **order by** *dept_name, name*
  - Example: **order by** *dept_name asc, name desc*

知乎帖子：

一个程序员的水平能差到什么程度：用 DB **order by** 操作对大数据文件进行排序

3.5 Set Operations

- **Set operations: union, intersect,** and **except** operate on relations
  - corresponding to relational algebra **operators** $\cup$, $\cap$, and $-$
    - **automatically eliminates duplicates**
    - **union all**, **intersect all** and **except all,** to retain duplicates
- **Find** courses in Fall 2017 **or** in Spring 2018

  (**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)

  union

  (**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)
- **Find** courses in Fall 2017 **and** in Spring 2018

  (**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)

  intersect

  (**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)

Set Operations

- **Find** courses in Fall 2017 **but not** in Spring 2018

(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)

except

(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)

3.6 Null Values

- **Null** signifies
  - an unknown value
  - a value does not exist
- The **result** of any arithmetic expression involving **null** is **null**
  - Example: 5 + **null** returns **null**
- The **predicate is null** can be used to check for null values.
  - Example: Find all instructors whose salary is null.

select *name*
from *instructor*
where *salary* **is null**

Null Values (Cont.)

- SQL treats as **unknown** the result of any comparison involving a **null** value
  - Example*: 5* < **null** or **null** <> **null** or **null** = **null**
- The predicate in a **where** clause can involve Boolean operations (**and**, **or**, **not**); thus Boolean operations need to be extended to deal with **unknown**.
  - **and** : *(true* **and** *unknown) = unknown,*
    (false **and** *unknown) = false,*
    (unknown **and** *unknown) = unknown*
  - **or:** (*unknown* **or** *true*) = *true*,
    (*unknown* **or** *false*) = *unknown*
    (*unknown* **or** *unknown) = unknown*

注意：
实际应用中避免引入属性 null ，防止索引失效；
用 default value 替代

- Result of **where** clause predicate is treated as **false** if it evaluates to **unknown**

3.7 Aggregate Functions

- **Aggregate functions operate** on values of **column**, but **return a value**

  avg: average value
  min: minimum value
  max: maximum value
  sum: sum of values
  count: number of values

- conduct **group by** subclause and **aggregate** function

  Select $\{A_1, A_2, ., A_i\}$, ***ag_fun***(A_{i+1}), …, ***ag_fun***(A_{i+k})

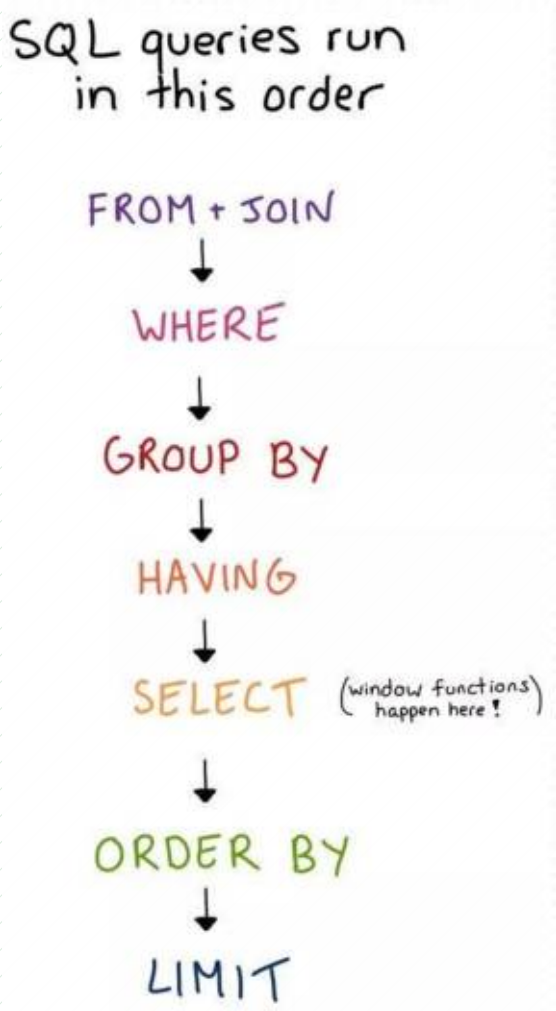
  From $r_1, r_2, \ldots, r_m$

  Where P_1

  { Group by $A_1, A_2, ., A_i$

  { **having** P_2 } **}**

  - P_1 is defined on all attributes of $r_1, r_2, \ldots, r_m$
  - P_2 is defined on $A_1, A_2, ., A_i$, as constraints **Group by**



Examples

- Find average salary of instructors in Computer Science department
  - **select avg** (*salary*)
    from *instructor*
    where *dept_name*= 'Comp. Sci.';

ID	*name*	*dept_name*	*salary*
45565	Jian	Comp. Sci	$100
74281	Ye	Comp. Sci	$250
98753	Du	Comp. Sci	$340
54123	Lin	Comp. Sci	$200

avg

- Find total number of instructors who teach a course in Spring 2018
  - **select count** (**distinct** *ID*)
    from *teaches*
    where *semester* = 'Spring' **and** *year* = 2018;
- Find the number of tuples in the ***course*** relation
  - **select count** (*)
    from *course*;

Aggregate Functions – Group By

- Find the average salary of instructors in **each department**
  - **select** *dept_name*, **avg** (*salary*) **as** *avg_salary*
    from *instructor*
    group by *dept_name*;

Aggregation (Cont.)

- **Attributes** in **select** clause **outside** of aggregate functions **must appear** in **group by** list
  - /* **erroneous query** */

    select *dept_name*, *ID*, **avg** (*salary*)
    from *instructor*
    group by *dept_name*

- Null values and aggregates
  - all aggregate operations **except count(*) ignore** tuples with *null* values on the aggregated attributes

Aggregate Functions – Having Clause

- Find the names and average salaries of all departments whose average salary is more than 42000

```
select dept_name, avg (salary) as avg_salary
from instructor
group by dept_name
    having avg (salary) > 42000;
```

- Note: predicates in **having** clause are applied **after** the formation of groups whereas predicates in **where** clause are applied **before** forming groups

3.8 Nested Subqueries

- **Subquery** is a **select-from-where** expression nested within another query.
- The nesting can be done in SQL query

select $A_1, A_2, \ldots, A_n$
from $r_1, r_2, \ldots, r_m$
where P

嵌套查询： 中间结果，便于构思化大为小， 化繁为简。 DBMS 实现时，要优化

注意：从 SQL 优化角度，避免使用低效 nested 查询，改为 from 子句中的多表联接，便于 DBMS 查询优化

- **from clause:** r_i can be replaced by any valid subquery
- **where clause:** P can be replaced with an expression:

  B <operation> (subquery)

  B is an attribute and <operation> to be defined later.
- **select clause:**

  A_i can be replaced be a subquery that generates a single value

Set Membership

- Find courses offered in Fall 2009 and in Spring 2010

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
        course_id in (select course_id
                      from section
                      where semester = 'Spring' and year= 2010);
```

- Find courses offered in Fall 2009 but not in Spring 2010

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
        course_id  not in (select course_id
                           from section
                           where semester = 'Spring' and year= 2010);
```

嵌入式查询执行过程

- 1. 从头至尾，依次扫描 *section* 中每一行；
- 2. 对 *section* 中每一行，
  - a. 判断其 semester、year 是否满足查询条件
  - b. 执行嵌入在 where 子句中的 *select* 查询，得到 10 年春季的 *course_id* 集合；
- 3. 判断本行的 *course_id* 是否在此集合中，决定本行的 *course_id* 是否所需查询结果

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Figure 2.6 The *section* relation.

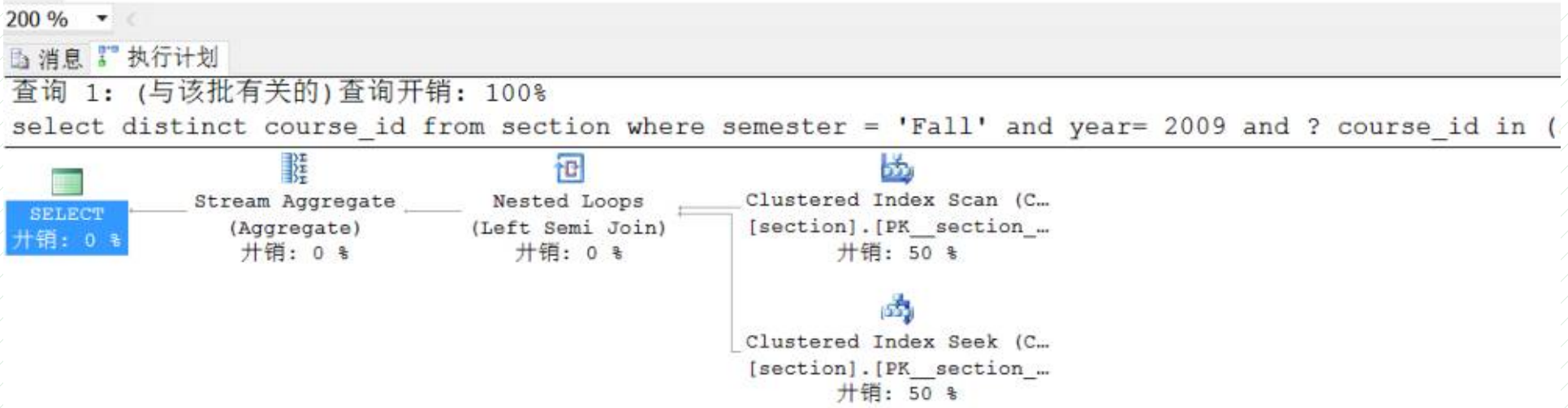
嵌入式查询执行过程

- 说明
  - 上述语句的有可能执行效率较低！
  - 被嵌入的 select 子句可能多次重复执行

- 实际应用中应尽量避免 membership 等嵌入式查询
  - 参见“数据库物理设计及查询优化”

嵌入式查询的查询执行计划

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
 course_id in (select course_id
               from section
               where semester = 'Spring' and year= 2010);
```



2 条查询结果相同、实现方式不一样的 SQL 多表查询

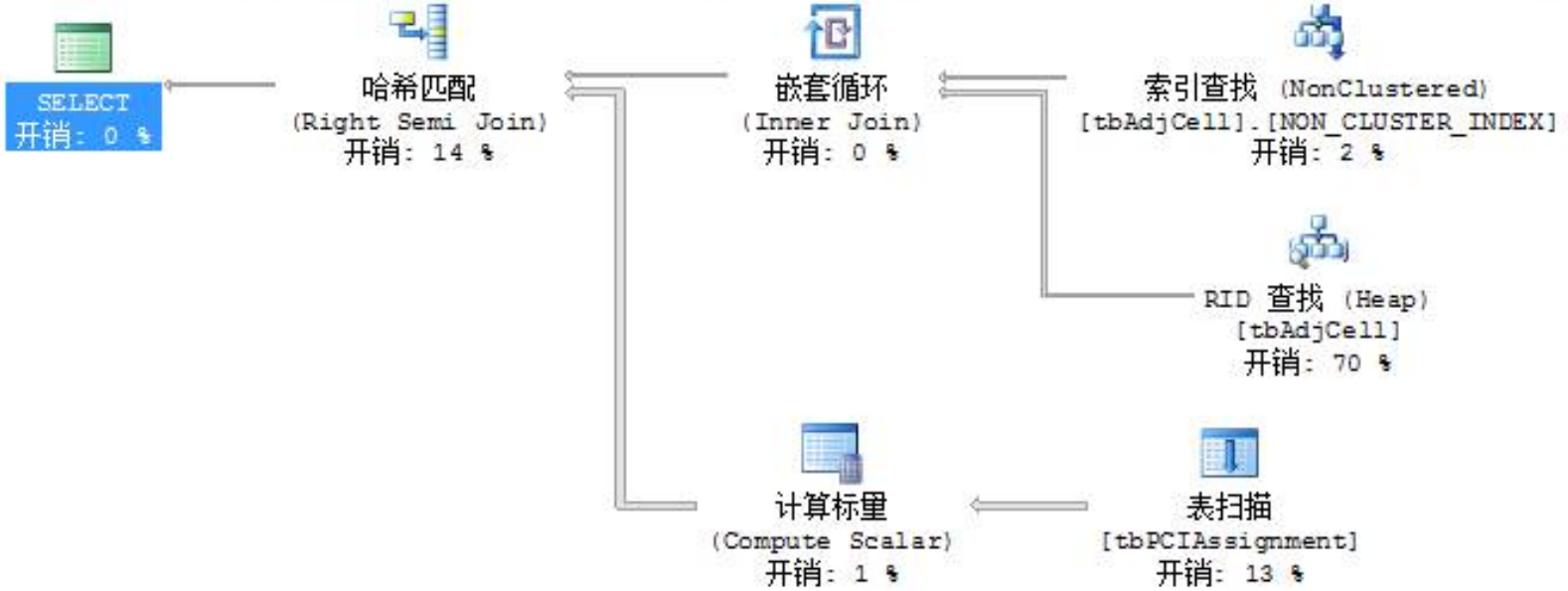
```
select SECTOR_ID,PHYCELLID
from   tbPCIAssignment
where  EARFCN='38400' and ASSIGN_ID=0
    and SECTOR_ID in
   (select N_SECTOR_ID
    from tbAdjCell
     where S_SECTOR_ID ='58617-1'
     and N_EARFCN='38400')
```

```
select A.SECTOR_ID,A.PHYCELLID
from tbPCIAssignment as A,
     tbAdjCell as B
where A.EARFCN='38400'
   and A.ASSIGN_ID=0
   and A.SECTOR_ID=B.N_SECTOR_ID
   and B.S_SECTOR_ID='58617-1'
```

SQL Server 下查询执行计划及执行时间比（ 77: 23 ）

查询 1: (与该批有关的)查询开销: 77%

```
select SECTOR_ID,PHYCELLID from tbPCIAssignment where EARFCN='38400' and ASSIGN_ID=0 and SECTOR_ID in (select N_SECTOR_ID from
```



2 次连接操作：
inner join 、 right semi join

查询 2: (与该批有关的)查询开销: 23%

```
select A.SECTOR_ID,A.PHYCELLID from tbPCIAssignment as A, tbAdjCell as B where B.S_SECTOR_ID='58617-1' and A.SECTOR_ID=B.N_SECT
```

1 次连接操作： inner join

Set Membership (Cont.)

- Name all instructors whose name is neither “Mozart” nor Einstein”

```
select distinct name
from instructor
where  name not in ('Mozart', 'Einstein')
```

- Find **the total number of (distinct) students** who have taken course sections taught by the instructor with ID 10101

takes, teaches

```
select count (distinct ID)
from takes
where (course_id, semester, year) in
        (select course_id, semester, year
         from teaches
         where teaches.ID= 10101)
```

语句的执行效率较低，why?!

被嵌入的 select 子句可能多次重复执行

Set Comparison – “some” Clause

- Find names of instructors with salary greater than that of some (at least one) instructor in Biology department.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept name = 'Biology';
```

- Same query using > **some** clause

优于

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept name = 'Biology');
```

Definition of "some" Clause

- F <comp> **some** $r \Leftrightarrow \exists\, t \in r$ such that (F <comp> t)
  Where <comp> can be: $<,\ \leq,\ >,\ =,\ \neq$

(5 < **some** | 0 / 5 / 6 |) = true (read: 5 < some tuple in the relation)

(5 < **some** | 0 / 5 |) = false

(5 = **some** | 0 / 5 |) = true

(5 ≠ **some** | 0 / 5 |) = true (since 0 ≠ 5)

- (= **some**) ≡ **in**
- However, (≠ **some**) ≢ **not in**

Set Comparison – “all” Clause

- Find the names of all instructors
- whose salary is more than the salary of all instructors in Biology department.

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept name = 'Biology')
```

Definition of "all" Clause

- F <comp> **all** $r \Leftrightarrow \forall\ t \in r$ (F <comp> *t)*

(5 < **all** | 0, 5, 6 |) = false

(5 < **all** | 6, 10 |) = true

(5 = **all** | 4, 5 |) = false

(5 ≠ **all** | 4, 6 |) = true (since 5 ≠ 4 and 5 ≠ 6)

- (≠ **all**) ≡ **not in**
- However, (= **all**) ≢ **in**

Test for Empty Relations

- The **exists** construct returns the value **true** if the argument subquery is nonempty
  - **exists** $r \Leftrightarrow r \neq \emptyset$
  - **not exists** $r \Leftrightarrow r = \emptyset$
- 集合包含
  - **except**: $X - Y$
  - $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
  - $X \subseteq Y \Leftrightarrow (X - Y) = \emptyset \Leftrightarrow$ **not exists (** X *except* Y **)**
- e.g. Find all ***students*** who take all the courses that takes by Li

  / 查询选修了“ Li ”所选修的全部课程的同学
  - 计算 Li 选修课程集合 X
  - 对每个学生，计算他选修的课程集合 Y
  - 判断 Y 是否包含 X

Use of exists Clause

- Find all courses taught in both Fall 2017 and Spring 2018

```
select course_id
from section as S
where semester = 'Fall' and year = 2017 and
        exists  (select *
                 from section as T
                 where semester = 'Spring' and year= 2018
                        and S.course_id = T.course_id);
```

- **Correlation name** – variable S in the outer query
- **Correlated subquery** – the inner query

Use of not exists Clause

- Find all students who have taken all courses offered in Biology department 

```
select distinct S.ID, S.name
from student as S
where not exists ( (select course_id
                    from course
                    where dept_name = 'Biology')
                   except
                    (select T.course_id
                     from takes as T
                     where S.ID = T.ID));
```

X : all *courses* offered in the Biology *department*

Y : all *courses* a particular student took

- Note that $X - Y = \emptyset \Leftrightarrow X \subseteq Y$

Test for Absence of Duplicate Tuples

- The **unique** tests whether a subquery has any duplicate tuples in result.
- Find all courses that were offered at most once in 2017

```
select T.course_id
from course as T
where unique (select R.course_id
                from section as R
                where T.course_id= R.course_id
                      and R.year = 2017)
```

Subqueries in From Clause

- Find **the average instructors' salaries** of those departments where the average salary is greater than $42,000."

```
select dept_name, avg_salary
 from ( select dept_name, avg (salary) as avg_salary
        from instructor
        group by dept_name)
 where avg_salary > 42000
```

- Another way:

```
select dept_name, avg_salary
from ( select dept_name, avg (salary)
       from instructor
       group by dept_name)
          as dept_avg (dept_name, avg_salary)
  where avg_salary > 42000
```

With Clause—— 建议优先使用

- The **with** clause provides a way of defining a **temporary** relation available only to the query in which the **with** clause occurs.
  - 将 1 个复杂查询分解为若干步，每个视图定义 1 个各步的中间计算结果，逻辑清晰
- Find all departments with the maximum budget

找出所有最高 budget 的 department（ 可能不只一个

```
with max_budget (value) as
        (select max(budget)
         from department)
select department.name
from department, max_budget
where department.budget = max_budget.value;
```

视图 max-budget(value) = {200000}, e.g.

定义局部视图

// 取子集作为中间逻辑结果

Complex Queries using With Clause

- Find departments where the total salary is greater than the average of the total salary at all departments

```
with dept _total (dept_name, value) as
        (select dept_name, sum(salary)
         from instructor
         group by dept_name ),
      dept_total_avg (value) as
       (select avg(value)
        from dept_total )
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```

Scalar Subquery

- Scalar subquery is for one where a single value is expected
- List all departments along with the number of instructors in each department

```
select dept_name,
        ( select count(*)
          from instructor
          where department.dept_name = instructor.dept_name )
          as num_instructors
from department
```

3.9 Modification of Database

- **Deletion** of **tuples** from a given relation.
- **Insertion** of new **tuples** into a given relation
- **Updating** of values in some **tuples** in a given relation

Deletion

- Delete all instructors

  delete from *instructor*

- Delete all instructors from Finance department

  delete from *instructor*
  where *dept_name*= 'Finance';

- Delete all tuples in instructor for those instructors associated with department located in Watson building.

  delete from *instructor*
  where *dept name* **in** (**select** *dept name*
  from *department*
  where *building* = 'Watson')

Deletion (Cont.)

- **Delete** all instructors whose salary is less than the average salary of instructors

```
delete from instructor
where salary < (select avg (salary)
                from instructor)
```

  - **Problem**: as deleting tuples from *instructor*, the average salary changes
  - Solution:
    - Firstly, compute **avg** (salary) and find all tuples to delete
    - Subsequently, delete all tuples found above

    (without recomputing **avg** or retesting the tuples)

Insertion

- **Add** a new tuple **to** *course*

```
insert into course
    values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- **Equivalently**

```
insert into course (course_id, title, dept_name, credits)
    values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- **Add** a new tuple to *student* with *tot_creds* set to null

```
insert into student
    values ('3003', 'Green', 'Finance', null);
```

Insertion (Cont.)

- **Make** each *student* in Music department who has earned more than 144 credit hours **an instructor** in the Music department **with salary of $18,000**.

```
insert into instructor
        select ID, name, dept_name, 18000
    from   student
    where   dept_name = 'Music' and total_cred > 144;
```

- The **select from where** statement is evaluated fully before any of its results are inserted into the relation.

  Otherwise queries like

  insert into *table*1 **select** * **from** *table*1

  would cause problem

Updates

- Give a 5% salary raise to all instructors

```
update instructor
    set salary = salary * 1.05
```

- Give a 5% salary raise to those instructors who earn less than 70000

```
update instructor
    set salary = salary * 1.05
    where salary < 70000;
```

- Give a 5% salary raise to instructors whose salary is less than average

```
update instructor
    set salary = salary * 1.05
    where salary <  (select avg (salary)
                     from instructor);
```

Updates (Cont.)

- Increase salaries of instructors whose salary is over $100,000 by 3%, and all others by a 5%
  - two **update** statements:

```
update instructor
    set salary = salary * 1.03
    where salary > 100,000;
update instructor
    set salary = salary * 1.05
    where salary <= 100,000;
```

(I)

(II)

  - the order is important
    - e.g. the statement order: (II); (I), resulting in twice increasing of salaries for the instructors with initial salary 99,999
  - can be done better using the **case** statement (next slide)

Case Statement for Conditional Updates

- Same query as before but with case statement

```
update instructor
set salary = case
                when salary <= 100000 then salary * 1.05
                else salary * 1.03
             end
```

Updates with Scalar Subqueries

- Recompute and update tot_creds value for all students

```
update student S
    set tot_cred = ( select sum(credits)
                     from takes, course
                     where takes.course_id = course.course_id  and
                         S.ID= takes.ID.and takes.grade <> 'F' and
                         takes.grade is not null );
```

- Sets *tot_creds* to null for students who have not taken any course
- Instead of **sum**(*credits*), use:

```
case
   when sum(credits) is not null then sum(credits)
   else 0
end
```

Update for Multiple Tables

- **Example**

  Given:

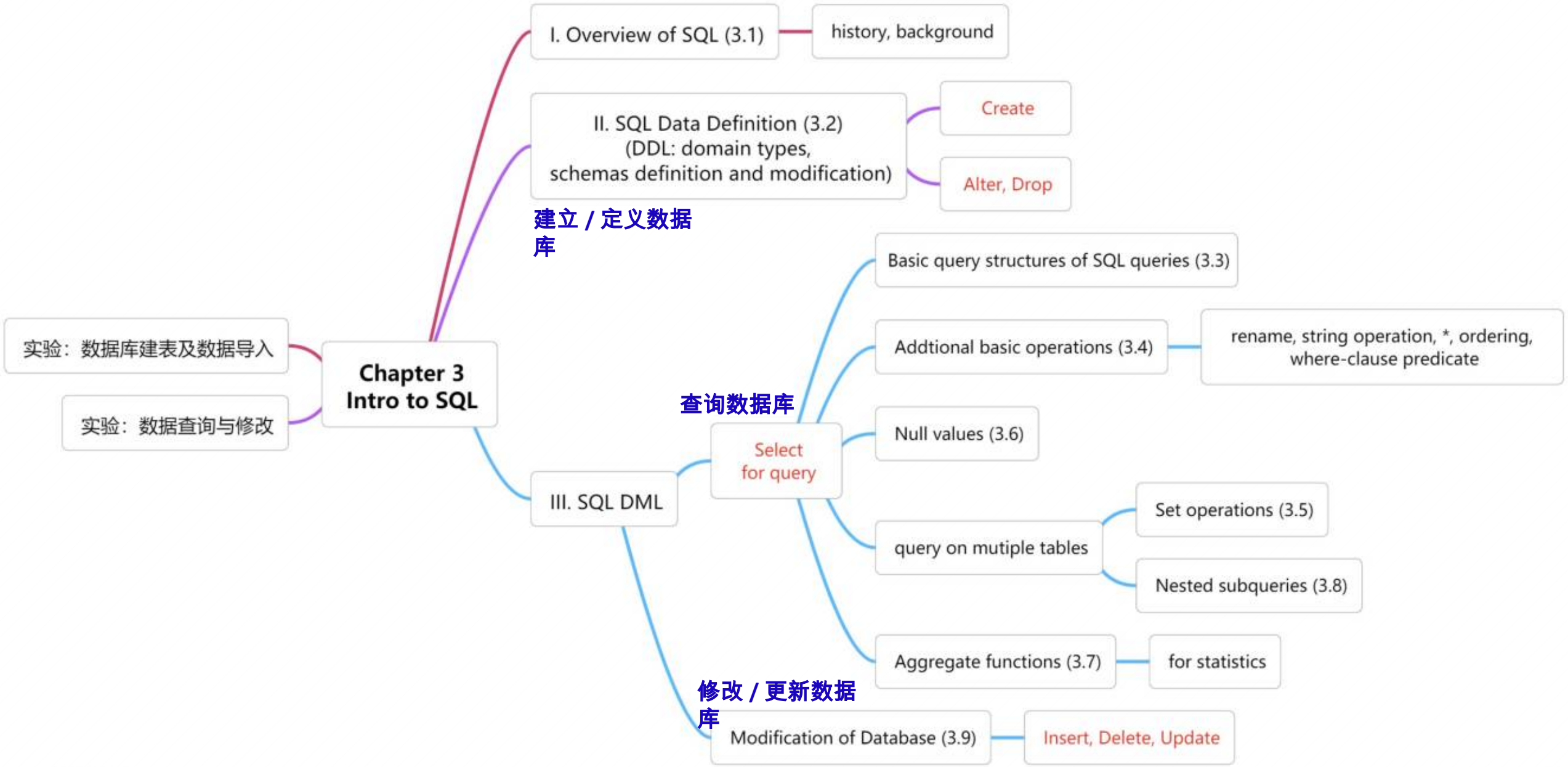
  Student (S#, Sname, age …, C#, Grade, …),

  SGrade (S#, C#, Grade)

  - **利用 *SGrade* 的内容，更新 *Student***

```
update   Student as A
set      Grade=B.Grade
from     SGrade as B
where    A.S#=B.S# and A.C#=B.C#
```

Outline


I. Overview of SQL (3.1)
history, background
II. SQL Data Definition (3.2)
(DDL: domain types,
schemas definition and modification)
Create
Alter, Drop
建立 / 定义数据库
实验：数据库建表及数据导入
实验：数据查询与修改
Chapter 3
Intro to SQL
III. SQL DML
查询数据库
Select
for query
Basic query structures of SQL queries (3.3)
Addtional basic operations (3.4)
rename, string operation, *, ordering,
where-clause predicate
Null values (3.6)
query on mutiple tables
Set operations (3.5)
Nested subqueries (3.8)
Aggregate functions (3.7)
for statistics
修改 / 更新数据库
Modification of Database (3.9)
Insert, Delete, Update

Thanks for your attention